

GYM Management System - Comprehensive Project Report

Table of Contents

- [1. Database Models](#)
- [2. API Routes](#)
- [3. Authentication Routes](#)
- [4. User Management Routes](#)
- [5. Subscription Management Routes](#)
- [6. Subscription Plan Routes](#)
- [7. Class Management Routes](#)
- [8. Attendance Routes](#)
- [9. Coach Assignment Routes](#)
- [10. Notification Routes](#)
- [11. Dashboard Routes](#)
- [12. User Flow Routes](#)
- [13. Admin Routes](#)
- [14. OAuth Routes](#)

Database Models

1. User Model

Table: users

Fields:

- id (INTEGER, PRIMARY KEY, AUTO INCREMENT)
- name (STRING(255), NOT NULL, 2-255 chars)
- email (STRING(255), NOT NULL, UNIQUE, VALID EMAIL)
- phone (STRING(20), NULLABLE, 10-20 chars)
- password_hash (STRING(255), NULLABLE)
- google_id (STRING(255), NULLABLE, UNIQUE)
- profile_picture (STRING(500), NULLABLE, VALID URL)
- role (ENUM: 'user', 'employee', 'admin', 'coach', DEFAULT: 'user')
- status (ENUM: 'pending_subscription', 'active', 'suspended', 'expired', 'frozen', DEFAULT: 'pending_subscription')
- email_verified (BOOLEAN, DEFAULT: false)
- email_verification_token (STRING(255), NULLABLE)
- email_verification_expires (DATE, NULLABLE)
- last_login (DATE, NULLABLE)
- created_at, updated_at (TIMESTAMPS)

Associations:

- Has many Subscriptions
- Has many Attendances
- Has many CoachAssignments (as user)
- Has many CoachAssignments (as coach)
- Has many Classes (as coach)
- Has many Notifications
- Has many SubscriptionAudits (as performer)

2. Subscription Model

Table: subscriptions

Fields:

- id (INTEGER, PRIMARY KEY, AUTO INCREMENT)
- user_id (INTEGER, NOT NULL, FOREIGN KEY → users.id)
- plan_id (INTEGER, NOT NULL, FOREIGN KEY → subscription_plans.id)
- payment_method (ENUM: 'cash', 'card')
- payment_status (ENUM: 'pending', 'paid', 'failed', DEFAULT: 'pending')
- confirmation_status (ENUM: 'pending', 'confirmed', 'rejected', DEFAULT: 'pending')
- confirmed_by (INTEGER, NULLABLE, FOREIGN KEY → users.id)
- start_date (DATEONLY, NULLABLE)
- end_date (DATEONLY, NULLABLE)
- frozen_until (DATEONLY, NULLABLE)
- frozen_reason (TEXT, NULLABLE)
- amount (DECIMAL(10,2), NOT NULL, MIN: 0)
- notes (TEXT, NULLABLE)
- created_at, updated_at (TIMESTAMPS)

Associations:

- Belongs to User
- Belongs to SubscriptionPlan
- Belongs to User (confirmed by)
- Has many SubscriptionAudits

3. SubscriptionPlan Model

Table: subscription_plans

Fields:

- id (INTEGER, PRIMARY KEY, AUTO INCREMENT)
- name (STRING(255), NOT NULL, 2-255 chars)
- description (TEXT, NULLABLE)
- duration_months (INTEGER, NOT NULL, 1-120)
- price (DECIMAL(10,2), NOT NULL, MIN: 0)
- features (JSONB, DEFAULT: [])
- is_active (BOOLEAN, DEFAULT: true)
- created_at, updated_at (TIMESTAMPS)

Associations:

- Has many Subscriptions

4. Class Model

Table: classes

Fields:

- id (INTEGER, PRIMARY KEY, AUTO INCREMENT)
- name (STRING(255), NOT NULL, 2-255 chars)

- description (TEXT, NULLABLE)
- coach_id (INTEGER, NULLABLE, FOREIGN KEY → users.id)
- capacity (INTEGER, DEFAULT: 20, 1-100)
- duration_minutes (INTEGER, DEFAULT: 60, 15-240)
- schedule_time (TIME, NULLABLE)
- schedule_days (JSONB, DEFAULT: [])
- price (DECIMAL(10,2), DEFAULT: 0.00, MIN: 0)
- registered_users (JSONB, DEFAULT: [])
- status (ENUM: 'available', 'full', 'cancelled', DEFAULT: 'available')
- is_active (BOOLEAN, DEFAULT: true)
- created_at (TIMESTAMP, NO updated_at)

Associations:

- Belongs to User (coach)

5. Attendance Model

Table: attendance

Fields:

- id (INTEGER, PRIMARY KEY, AUTO INCREMENT)
- user_id (INTEGER, NOT NULL, FOREIGN KEY → users.id)
- check_in_time (DATE, DEFAULT: NOW)
- check_out_time (DATE, NULLABLE)
- recorded_by (INTEGER, NULLABLE, FOREIGN KEY → users.id)
- notes (TEXT, NULLABLE)
- created_at (TIMESTAMP, NO updated_at)

Associations:

- Belongs to User (attendee)
- Belongs to User (recorded by)

6. Notification Model

Table: notifications

Fields:

- id (INTEGER, PRIMARY KEY, AUTO INCREMENT)
- user_id (INTEGER, NOT NULL, FOREIGN KEY → users.id)
- title (STRING(255), NOT NULL, 1-255 chars)
- message (TEXT, NOT NULL)
- type (ENUM: 'subscription', 'payment', 'class', 'general', 'email_verification', DEFAULT: 'general')
- is_read (BOOLEAN, DEFAULT: false)
- created_at (TIMESTAMP, NO updated_at)

Associations:

- Belongs to User

7. CoachAssignment Model

Table: coach_assignments

Fields:

- id (INTEGER, PRIMARY KEY, AUTO INCREMENT)
- coach_id (INTEGER, NOT NULL, FOREIGN KEY → users.id)
- user_id (INTEGER, NOT NULL, FOREIGN KEY → users.id)
- assigned_date (DATEONLY, DEFAULT: NOW)
- is_active (BOOLEAN, DEFAULT: true)
- notes (TEXT, NULLABLE)
- NO TIMESTAMPS

Associations:

- Belongs to User (user)
- Belongs to User (coach)

8. SubscriptionAudit Model

Table: subscription_audit

Fields:

- id (INTEGER, PRIMARY KEY, AUTO INCREMENT)
- subscription_id (INTEGER, NOT NULL, FOREIGN KEY → subscriptions.id)
- action (ENUM: 'created', 'confirmed', 'rejected', 'modified', 'frozen', 'unfrozen')
- performed_by (INTEGER, NOT NULL, FOREIGN KEY → users.id)
- old_status (STRING(50), NULLABLE)
- new_status (STRING(50), NULLABLE)
- notes (TEXT, NULLABLE)
- created_at (TIMESTAMP, NO updated_at)

Associations:

- Belongs to Subscription
- Belongs to User (performed by)

API Routes

Authentication Routes (/api/auth)

1. POST /api/auth/register

Purpose: Register new user account

Input:

```
{
  "name": "string (required, 2-255 chars)",
  "email": "string (required, valid email)",
  "phone": "string (optional, 10-20 chars)",
  "password": "string (required)",
  "role": "string (optional, default: 'user')"
```

Output:

```
{
  "success": true,
  "message": "Registration successful! Please check your email to verify your account.",
  "token": "jwt_token_string",
  "user": {
    "id": "number",
    "name": "string",
    "email": "string",
    "phone": "string",
    "role": "string",
    "status": "string",
    "email_verified": "boolean"
  }
}
```

Rate Limiting: Register limiter

Authentication: None required

Side Effects:

- Sends email verification email (async)
- Creates welcome notification (async)
- Sets HTTP-only auth cookie

2. POST /api/auth/login

Purpose: Authenticate existing user

Input:

```
{
  "email": "string (required, valid email)",
  "password": "string (required)"
}
```

Output:

```
{
  "success": true,
  "message": "Login successful",
  "token": "jwt_token_string",
  "user": {
    "id": "number",
    "name": "string",
    "email": "string",
    "phone": "string",
    "role": "string",
    "status": "string",
    "email_verified": "boolean",
    "last_login": "datetime"
  }
}
```

Rate Limiting: Auth limiter

Authentication: None required

Side Effects:

- Updates last_login timestamp
- Sets HTTP-only auth cookie

3. POST /api/auth/logout

Purpose: Clear authentication token

Input: None

Output:

```
{
  "success": true,
  "message": "Logout successful"
}
```

Rate Limiting: None

Authentication: None required

Side Effects: Clears HTTP-only auth cookie

4. GET /api/auth/verify-email/token

Purpose: Verify user email address

Input: URL parameter token (string)

Output:

```
{
  "success": true,
  "message": "Email verified successfully!"
}
```

Rate Limiting: Email verification limiter

Authentication: None required

Side Effects:

- Sets email_verified to true
- Clears verification token
- Updates last_login
- Sends welcome email (async)
- Creates verification notification (async)

5. GET /api/auth/profile

Purpose: Get current authenticated user profile

Input: None (uses JWT token)

Output:

```
{
  "success": true,
  "user": {
    "id": "number",
    "name": "string",
    "email": "string",
    "phone": "string",
    "role": "string",
    "status": "string",
    "email_verified": "boolean",
    "last_login": "datetime",
    "created_at": "datetime",
    "updated_at": "datetime"
  }
}
```

Rate Limiting: None

Authentication: JWT token required

User Management Routes (/api/users)

1. GET /api/users

Purpose: Get all users (admin/employee only)

Input:

- Query parameters: page, limit, role, status, search

Output:

```
{
  "success": true,
  "data": {
    "users": [
      {
        "id": "number",
        "name": "string",
        "email": "string",
        "phone": "string",
        "role": "string",
        "status": "string",
        "email_verified": "boolean",
        "created_at": "datetime",
        "subscriptions": []
      }
    ],
    "pagination": {
      "currentPage": "number",
      "totalPages": "number",
      "totalUsers": "number",
      "limit": "number"
    }
  }
}
```

Authentication: JWT + role('admin', 'employee')

2. GET /api/users/:id

Purpose: Get specific user by ID (admin/employee only)

Input: URL parameter id (number)

Output:

```
{
  "success": true,
  "data": {
    "user": {
      "id": "number",
      "name": "string",
      "email": "string",
      "phone": "string",
      "role": "string",
      "status": "string",
      "email_verified": "boolean",
      "created_at": "datetime",
      "subscriptions": [],
      "attendances": []
    }
  }
}
```

Authentication: JWT + role('admin', 'employee')

3. PUT /api/users/:id

Purpose: Update user information (admin only)

Input:

- URL parameter id (number)
- Body:

```
{
  "name": "string (optional)",
  "email": "string (optional)",
  "phone": "string (optional)",
  "role": "string (optional)",
  "status": "string (optional)"
}
```

Output:

```
{
  "success": true,
  "message": "User updated successfully",
  "data": {
    "user": {
      "id": "number",
      "name": "string",
      "email": "string",
      "phone": "string",
      "role": "string",
      "status": "string"
    }
  }
}
```

Authentication: JWT + role('admin')

4. DELETE /api/users/id

Purpose: Delete user account (admin only)

Input: URL parameter id (number)

Output:

```
{
  "success": true,
  "message": "User deleted successfully"
}
```

Authentication: JWT + role('admin')

Subscription Management Routes (/api/subscriptions)

1. GET /api/subscriptions/my

Purpose: Get current user's subscriptions

Input: None (uses JWT)

Output:

```
{
  "success": true,
  "data": {
    "subscriptions": [
      {
        "id": "number",
        "payment_method": "string",
        "payment_status": "string",
        "confirmation_status": "string",
        "start_date": "date",
        "end_date": "date",
        "frozen_until": "date",
        "frozen_reason": "string",
        "amount": "decimal",
        "notes": "string",
        "plan": {
          "name": "string",
          "duration_months": "number",
          "price": "decimal"
        }
      }
    ]
  }
}
```

Authentication: JWT required

2. POST /api/subscriptions

Purpose: Create new subscription

Input:

```
{
  "plan_id": "number (required)",
  "payment_method": "string (required, 'cash' or 'card')",
  "notes": "string (optional)"
}
```

Output:

```
{
  "success": true,
  "message": "Subscription created successfully",
  "data": {
    "subscription": {
      "id": "number",
      "user_id": "number",
      "plan_id": "number",
      "payment_method": "string",
      "payment_status": "pending",
      "confirmation_status": "pending",
      "amount": "decimal"
    }
  }
}
```

Authentication: JWT required

Side Effects: Creates subscription audit log

3. GET /api/subscriptions

Purpose: Get all subscriptions (admin/employee)

Input: Query parameters: page, limit, status, payment_status, user_id

Output:

```
{
  "success": true,
  "data": {
    "subscriptions": [
      {
        "id": "number",
        "payment_method": "string",
        "payment_status": "string",
        "confirmation_status": "string",
        "amount": "decimal",
        "user": {
          "name": "string",
          "email": "string"
        },
        "plan": {
          "name": "string",
          "price": "decimal"
        }
      }
    ],
    "pagination": {
      "currentPage": "number",
      "totalPages": "number",
      "totalSubscriptions": "number"
    }
  }
}
```

Authentication: JWT + role('admin','employee')

4. GET /api/subscriptions/id

Purpose: Get specific subscription details

Input: URL parameter id (number)

Output:

```
{
  "success": true,
  "data": {
    "subscription": {
      "id": "number",
      "payment_method": "string",
      "payment_status": "string",
      "confirmation_status": "string",
      "start_date": "date",
      "end_date": "date",
      "amount": "decimal",
      "user": {},
      "plan": {},
      "audits": []
    }
  }
}
```

Authentication: JWT + role('admin','employee')

5. PATCH /api/subscriptions/id/confirm

Purpose: Confirm subscription (admin/employee)

Input:

- URL parameter id (number)
- Body:

```
{
  "start_date": "date (optional)",
  "notes": "string (optional)"
}
```

Output:

```
{
  "success": true,
  "message": "Subscription confirmed successfully"
}
```

Authentication: JWT + role('admin','employee')

Side Effects:

- Updates user status to 'active'
- Creates audit log
- Creates notification

6. PATCH /api/subscriptions/id/reject

Purpose: Reject subscription (admin/employee)

Input:

- URL parameter id (number)
- Body:

```
{
  "reason": "string (required)"
}
```

Output:

```
{
  "success": true,
  "message": "Subscription rejected successfully"
}
```

Authentication: JWT + role('admin','employee')

Side Effects: Creates audit log and notification

7. PATCH /api/subscriptions/id/freeze

Purpose: Freeze subscription temporarily

Input:

- URL parameter `id` (number)
- Body:

```
{
  "freeze_until": "date (required)",
  "reason": "string (required)"
}
```

Output:

```
{
  "success": true,
  "message": "Subscription frozen successfully"
}
```

Authentication: JWT + role('admin', 'employee')

Side Effects: Updates user status, creates audit log

8. PATCH /api/subscriptions/:id/unfreeze

Purpose: Unfreeze subscription

Input: URL parameter `id` (number)

Output:

```
{
  "success": true,
  "message": "Subscription unfrozen successfully"
}
```

Authentication: JWT + role('admin', 'employee')

Side Effects: Restores user status, creates audit log

9. PATCH /api/subscriptions/:id/cancel

Purpose: Cancel own subscription

Input:

- URL parameter `id` (number)
- Body:

```
{
  "reason": "string (optional)"
}
```

Output:

```
{
  "success": true,
  "message": "Subscription cancelled successfully"
}
```

Authentication: JWT required (own subscription only)

Subscription Plan Routes (/api/subscription-plans)

1. GET /api/subscription-plans

Purpose: Get all active subscription plans

Input: Query parameters: `active` (boolean, default: true)

Output:

```
{
  "success": true,
  "data": {
    "plans": [
      {
        "id": "number",
        "name": "string",
        "description": "string",
        "duration_months": "number",
        "price": "decimal",
        "features": "array",
        "is_active": "boolean"
      }
    ]
  }
}
```

Authentication: None required

2. GET /api/subscription-plans/:id

Purpose: Get specific plan details

Input: URL parameter `id` (number)

Output:

```
{
  "success": true,
  "data": {
    "plan": {
      "id": "number",
      "name": "string",
      "description": "string",
      "duration_months": "number",
      "price": "decimal",
      "features": "array",
      "is_active": "boolean"
    }
  }
}
```

Authentication: None required

3. POST /api/subscription-plans

Purpose: Create new subscription plan (admin only)

Input:

```
{
  "name": "string (required, 2-255 chars)",
  "description": "string (optional)",
  "duration_months": "number (required, 1-120)",
  "price": "number (required, min: 0)",
  "features": "array (optional)"
}
```

Output:

```
{
  "success": true,
  "message": "Subscription plan created successfully",
  "data": {
    "plan": {
      "id": "number",
      "name": "string",
      "description": "string",
      "duration_months": "number",
      "price": "decimal",
      "features": "array"
    }
  }
}
```

Authentication: JWT + role('admin')

4. PUT /api/subscription-plans/:id

Purpose: Update subscription plan (admin only)

Input:

- URL parameter `id` (number)
- Body: Same as create, all fields optional

Output:

```
{
  "success": true,
  "message": "Subscription plan updated successfully",
  "data": {
    "plan": {
      "id": "number",
      "name": "string",
      "description": "string",
      "duration_months": "number",
      "price": "decimal",
      "features": "array"
    }
  }
}
```

Authentication: JWT + role('admin')

5. DELETE /api/subscription-plans/:id

Purpose: Deactivate subscription plan (admin only)

Input: URL parameter `id` (number)

Output:

```
{
  "success": true,
  "message": "Subscription plan deactivated successfully"
}
```

Authentication: JWT + role('admin')

Note: Soft delete - sets `is_active` to false

Class Management Routes (/api/classes)

1. GET /api/classes

Purpose: Get all classes

Input: Query parameters: `active`, `coach_id`, `page`, `limit`

Output:


```
{
  "success": true,
  "data": {
    "classes": [
      {
        "id": "number",
        "name": "string",
        "description": "string",
        "capacity": "number",
        "duration_minutes": "number",
        "schedule_time": "time",
        "schedule_days": "array",
        "price": "decimal",
        "status": "string",
        "registered_count": "number",
        "coach": {
          "name": "string",
          "email": "string"
        }
      }
    ],
    "pagination": {
      "currentPage": "number",
      "totalPages": "number",
      "totalClasses": "number"
    }
  }
}
```

Authentication: None required

2. GET /api/classes/id

Purpose: Get specific class details

Input: URL parameter `id` (number)

Output:

```
{
  "success": true,
  "data": {
    "class": {
      "id": "number",
      "name": "string",
      "description": "string",
      "capacity": "number",
      "duration_minutes": "number",
      "schedule_time": "time",
      "schedule_days": "array",
      "price": "decimal",
      "registered_users": "array",
      "status": "string",
      "coach": {}
    }
  }
}
```

Authentication: None required

3. POST /api/classes

Purpose: Create new class (admin only)

Input:

```
{
  "name": "string (required, 2-255 chars)",
  "description": "string (optional)",
  "coach_id": "number (optional)",
  "capacity": "number (optional, 1-100, default: 20)",
  "duration_minutes": "number (optional, 15-240, default: 60)",
  "schedule_time": "time (optional)",
  "schedule_days": "array (optional)",
  "price": "number (optional, min: 0, default: 0)"
}
```

Output:

```
{
  "success": true,
  "message": "Class created successfully",
  "data": {
    "class": {
      "id": "number",
      "name": "string",
      "description": "string",
      "capacity": "number",
      "duration_minutes": "number",
      "price": "decimal"
    }
  }
}
```

Authentication: JWT + role('admin')

4. PUT /api/classes/id

Purpose: Update class (admin only)

Input:

- URL parameter `id` (number)
- Body: Same as create, all fields optional

Output:

```
{
  "success": true,
  "message": "Class updated successfully",
  "data": {
    "class": {}
  }
}
```

Authentication: JWT + role('admin')

5. DELETE /api/classes/id

Purpose: Deactivate class (admin only)

Input: URL parameter id (number)

Output:

```
{
  "success": true,
  "message": "Class deactivated successfully"
}
```

Authentication: JWT + role('admin')

6. POST /api/classes/id/register

Purpose: Register for a class

Input: URL parameter id (number)

Output:

```
{
  "success": true,
  "message": "Successfully registered for class"
}
```

Authentication: JWT required

Side Effects: Adds user to class registered_users array

7. DELETE /api/classes/id/unregister

Purpose: Unregister from a class

Input: URL parameter id (number)

Output:

```
{
  "success": true,
  "message": "Successfully unregistered from class"
}
```

Authentication: JWT required

Side Effects: Removes user from class registered_users array

Attendance Routes (/api/attendance)

1. GET /api/attendance/my

Purpose: Get current user's attendance history

Input: Query parameters: start_date, end_date, page, limit

Output:

```
{
  "success": true,
  "data": {
    "attendances": [
      {
        "id": "number",
        "check_in_time": "datetime",
        "check_out_time": "datetime",
        "duration_minutes": "number",
        "notes": "string",
        "recordedBy": {
          "name": "string"
        }
      }
    ],
    "pagination": {},
    "stats": {
      "totalVisits": "number",
      "totalHours": "number",
      "averageSessionTime": "number"
    }
  }
}
```

Authentication: JWT required

2. POST /api/attendance/check-in

Purpose: Check in to gym

Input:

```
{
  "notes": "string (optional)"
}
```

Output:

```
{
  "success": true,
  "message": "Checked in successfully",
  "data": {
    "attendance": {
      "id": "number",
      "check_in_time": "datetime",
      "notes": "string"
    }
  }
}
```

Authentication: JWT required

3. PATCH /api/attendance/check-out

Purpose: Check out from gym

Input:

```
{
  "notes": "string (optional)"
}
```

Output:

```
{
  "success": true,
  "message": "Checked out successfully",
  "data": {
    "attendance": {
      "id": "number",
      "check_in_time": "datetime",
      "check_out_time": "datetime",
      "duration_minutes": "number"
    }
  }
}
```

Authentication: JWT required

4. GET /api/attendance

Purpose: Get all attendance records (admin/employee)

Input: Query parameters: user_id, start_date, end_date, page, limit

Output:

```
{
  "success": true,
  "data": {
    "attendances": [
      {
        "id": "number",
        "check_in_time": "datetime",
        "check_out_time": "datetime",
        "duration_minutes": "number",
        "user": {
          "name": "string",
          "email": "string"
        },
        "recordedBy": {
          "name": "string"
        }
      }
    ],
    "pagination": {},
    "summary": {
      "totalRecords": "number",
      "activeUsers": "number"
    }
  }
}
```

Authentication: JWT + role('admin', 'employee')

5. POST /api/attendance/manual-check-in

Purpose: Manual check-in for user (employee/admin)

Input:

```
{
  "user_id": "number (required)",
  "check_in_time": "datetime (optional, default: now)",
  "notes": "string (optional)"
}
```

Output:

```
{
  "success": true,
  "message": "User checked in manually",
  "data": {
    "attendance": {}
  }
}
```

Authentication: JWT + role('admin', 'employee')

6. PATCH /api/attendance/:id/manual-check-out

Purpose: Manual check-out for user (employee/admin)

Input:

- URL parameter id (number)
- Body:

```
{
  "check_out_time": "datetime (optional, default: now)",
  "notes": "string (optional)"
}
```

Output:

```
{
  "success": true,
  "message": "User checked out manually"
}
```

Authentication: JWT + role('admin','employee')

Coach Assignment Routes (/api/coach-assignments)

1. GET /api/coach-assignments/my-clients

Purpose: Get clients assigned to current coach

Input: Query parameters: active, page, limit

Output:

```
{
  "success": true,
  "data": {
    "assignments": [
      {
        "id": "number",
        "assigned_date": "date",
        "is_active": "boolean",
        "notes": "string",
        "user": {
          "id": "number",
          "name": "string",
          "email": "string",
          "status": "string"
        }
      }
    ],
    "pagination": {},
    "stats": {
      "totalClients": "number",
      "activeClients": "number"
    }
  }
}
```

Authentication: JWT + role('coach')

2. GET /api/coach-assignments/my-coach

Purpose: Get coach assigned to current user

Input: None

Output:

```
{
  "success": true,
  "data": {
    "assignment": {
      "id": "number",
      "assigned_date": "date",
      "is_active": "boolean",
      "notes": "string",
      "coach": {
        "id": "number",
        "name": "string",
        "email": "string"
      }
    }
  }
}
```

Authentication: JWT required

3. GET /api/coach-assignments

Purpose: Get all coach assignments (admin/employee)

Input: Query parameters: coach_id, user_id, active, page, limit

Output:

```
{
  "success": true,
  "data": {
    "assignments": [
      {
        "id": "number",
        "assigned_date": "date",
        "is_active": "boolean",
        "coach": {
          "name": "string",
          "email": "string"
        },
        "user": {
          "name": "string",
          "email": "string"
        }
      }
    ],
    "pagination": {}
  }
}
```

Authentication: JWT + role('admin','employee')

4. POST /api/coach-assignments

Purpose: Create coach assignment (admin/employee)

Input:

```
{
  "coach_id": "number (required)",
  "user_id": "number (required)",
  "notes": "string (optional)"
}
```

Output:

```
{
  "success": true,
  "message": "Coach assignment created successfully",
  "data": {
    "assignment": {
      "id": "number",
      "coach_id": "number",
      "user_id": "number",
      "assigned_date": "date",
      "notes": "string"
    }
  }
}
```

Authentication: JWT + role('admin','employee')

5. PUT /api/coach-assignments/id

Purpose: Update coach assignment (admin/employee)

Input:

- URL parameter id (number)
- Body:

```
{
  "coach_id": "number (optional)",
  "is_active": "boolean (optional)",
  "notes": "string (optional)"
}
```

Output:

```
{
  "success": true,
  "message": "Coach assignment updated successfully"
}
```

Authentication: JWT + role('admin','employee')

6. DELETE /api/coach-assignments/id

Purpose: Deactivate coach assignment (admin/employee)

Input: URL parameter id (number)

Output:

```
{
  "success": true,
  "message": "Coach assignment deactivated successfully"
}
```

Authentication: JWT + role('admin','employee')

Notification Routes (/api/notifications)

1. GET /api/notifications

Purpose: Get current user's notifications

Input: Query parameters: type, is_read, page, limit

Output:

```
{
  "success": true,
  "data": {
    "notifications": [
      {
        "id": "number",
        "title": "string",
        "message": "string",
        "type": "string",
        "is_read": "boolean",
        "created_at": "datetime"
      }
    ],
    "pagination": {},
    "stats": {
      "unreadCount": "number",
      "totalCount": "number"
    }
  }
}
```

Authentication: JWT required

2. PATCH /api/notifications/id/read

Purpose: Mark notification as read

Input: URL parameter id (number)

Output:

```
{
  "success": true,
  "message": "Notification marked as read"
}
```

Authentication: JWT required (own notification only)

3. PATCH /api/notifications/mark-all-read

Purpose: Mark all notifications as read

Input: None

Output:

```
{
  "success": true,
  "message": "All notifications marked as read"
}
```

Authentication: JWT required

4. DELETE /api/notifications/:id

Purpose: Delete specific notification

Input: URL parameter id (number)

Output:

```
{
  "success": true,
  "message": "Notification deleted successfully"
}
```

Authentication: JWT required (own notification only)

5. POST /api/notifications/send

Purpose: Send notification to user(s) (admin/employee)

Input:

```
{
  "user_id": "number (optional, if not provided, sends to all)",
  "title": "string (required)",
  "message": "string (required)",
  "type": "string (optional, default: 'general')"
}
```

Output:

```
{
  "success": true,
  "message": "Notification(s) sent successfully",
  "data": {
    "sentCount": "number"
  }
}
```

Authentication: JWT + role('admin','employee')

Dashboard Routes (/api/dashboard)

1. GET /api/dashboard/user

Purpose: Get user dashboard data

Input: None

Output:

```
{
  "success": true,
  "data": {
    "user": {
      "name": "string",
      "status": "string",
      "email_verified": "boolean"
    },
    "subscription": {
      "status": "string",
      "plan_name": "string",
      "end_date": "date",
      "days_remaining": "number"
    },
    "attendance": {
      "thisMonth": "number",
      "lastMonth": "number",
      "totalHours": "number",
      "averagePerWeek": "number"
    },
    "classes": {
      "registered": "array",
      "upcoming": "array"
    },
    "coach": {
      "name": "string",
      "email": "string"
    },
    "notifications": {
      "unread": "array",
      "recent": "array"
    }
  }
}
```

Authentication: JWT + role('user')

2. GET /api/dashboard/employee

Purpose: Get employee dashboard data
Input: None
Output:

```
{
  "success": true,
  "data": {
    "stats": {
      "totalUsers": "number",
      "activeSubscriptions": "number",
      "pendingSubscriptions": "number",
      "todayAttendance": "number"
    },
    "subscriptions": {
      "recent": "array",
      "pendingConfirmation": "array"
    },
    "users": {
      "newThisWeek": "number",
      "expiringSubscriptions": "array"
    },
    "attendance": {
      "today": "number",
      "thisWeek": "number",
      "peakHours": "object"
    }
  }
}
```

Authentication: JWT + role('employee')

3. GET /api/dashboard/admin

Purpose: Get admin dashboard data
Input: None
Output:

```
{
  "success": true,
  "data": {
    "overview": {
      "totalUsers": "number",
      "totalRevenue": "number",
      "activeSubscriptions": "number",
      "totalClasses": "number"
    },
    "revenue": {
      "thisMonth": "number",
      "lastMonth": "number",
      "growth": "number"
    },
    "users": {
      "newThisMonth": "number",
      "activeUsers": "number",
      "churnRate": "number"
    },
    "subscriptions": {
      "pending": "number",
      "confirmed": "number",
      "expired": "number"
    },
    "attendance": {
      "todayTotal": "number",
      "weeklyAverage": "number",
      "popularDays": "array"
    },
    "staff": {
      "totalEmployees": "number",
      "totalCoaches": "number"
    }
  }
}
```

Authentication: JWT + role('admin')

User Flow Routes (/api/user-flow)

1. GET /api/user-flow/profile

Purpose: Get user profile for editing
Input: None
Output:

```
{
  "success": true,
  "data": {
    "profile": {
      "id": "number",
      "name": "string",
      "email": "string",
      "phone": "string",
      "role": "string",
      "status": "string",
      "email_verified": "boolean",
      "created_at": "datetime"
    }
  }
}
```

Authentication: JWT required

2. PUT /api/user-flow/profile

Purpose: Update user profile (basic info only)
Input:

```
{
  "name": "string (optional)",
  "phone": "string (optional)"
}
```

Output:

```
{
  "success": true,
  "message": "Profile updated successfully",
  "data": {
    "profile": {
      "id": "number",
      "name": "string",
      "phone": "string"
    }
  }
}
```

Authentication: JWT required
Note: Email updates require admin action

3. POST /api/user-flow/change-password

Purpose: Change user password
Input:

```
{
  "current_password": "string (required)",
  "new_password": "string (required)"
}
```

Output:

```
{
  "success": true,
  "message": "Password changed successfully"
}
```

Authentication: JWT required

4. GET /api/user-flow/activity-history

Purpose: Get user's activity history
Input: Query parameters: type, start_date, end_date, page, limit
Output:

```
{
  "success": true,
  "data": {
    "activities": [
      {
        "id": "number",
        "type": "string",
        "description": "string",
        "date": "datetime",
        "details": "object"
      }
    ],
    "pagination": {},
    "summary": {
      "totalActivities": "number",
      "recentActivity": "datetime"
    }
  }
}
```

Authentication: JWT required

5. GET /api/user-flow/subscription-status

Purpose: Get detailed subscription status
Input: None
Output:

```
{
  "success": true,
  "data": {
    "currentSubscription": {
      "id": "number",
      "status": "string",
      "start_date": "date",
      "end_date": "date",
      "days_remaining": "number",
      "plan": {
        "name": "string",
        "price": "decimal",
        "features": "array"
      }
    },
    "subscriptionHistory": "array",
    "availablePlans": "array",
    "canUpgrade": "boolean"
  }
}
```

Authentication: JWT required

Admin Routes (/api/admin)

1. GET /api/admin/settings

Purpose: Get system settings (NOT IMPLEMENTED)

Input: None

Output:

```
{
  "success": false,
  "message": "System settings functionality not implemented - requires Settings model"
}
```

Authentication: JWT + role('admin')

2. PUT /api/admin/settings

Purpose: Update system settings (NOT IMPLEMENTED)

Input: Any

Output:

```
{
  "success": false,
  "message": "System settings functionality not implemented - requires Settings model"
}
```

Authentication: JWT + role('admin')

3. GET /api/admin/user-management

Purpose: Get comprehensive user management data

Input: Query parameters: page, limit, role, status, search

Output:

```
{
  "success": true,
  "data": {
    "users": [
      {
        "id": "number",
        "name": "string",
        "email": "string",
        "role": "string",
        "status": "string",
        "last_login": "datetime",
        "subscription_status": "string",
        "total_spent": "decimal"
      }
    ],
    "pagination": {},
    "summary": {
      "totalUsers": "number",
      "activeUsers": "number",
      "newThisMonth": "number"
    }
  }
}
```

Authentication: JWT + role('admin')

4. POST /api/admin/users/bulk-action

Purpose: Perform bulk actions on users

Input:

```
{
  "user_ids": "array (required)",
  "action": "string (required: 'activate', 'suspend', 'delete', 'send_notification')",
  "data": "object (optional, additional data based on action)"
}
```

Output:

```
{
  "success": true,
  "message": "Bulk action completed successfully",
  "data": {
    "processed": "number",
    "failed": "number",
    "results": "array"
  }
}
```

Authentication: JWT + role('admin')

5. GET /api/admin/staff

Purpose: Get staff management data

Input: Query parameters: role, active, page, limit

Output:

```
{
  "success": true,
  "data": {
    "staff": [
      {
        "id": "number",
        "name": "string",
        "email": "string",
        "role": "string",
        "last_login": "datetime",
        "assigned_clients": "number"
      }
    ],
    "pagination": {},
    "summary": {
      "totalStaff": "number",
      "coaches": "number",
      "employees": "number"
    }
  }
}
```

Authentication: JWT + role('admin')

6. GET /api/admin/reports

Purpose: Get system reports

Input: Query parameters: type, start_date, end_date, format

Output:

```
{
  "success": true,
  "data": {
    "reports": {
      "revenue": "object",
      "attendance": "object",
      "subscriptions": "object",
      "user_activity": "object"
    },
    "generatedAt": "datetime"
  }
}
```

Authentication: JWT + role('admin')

7. GET /api/admin/analytics

Purpose: Get detailed analytics

Input: Query parameters: period, metrics

Output:

```
{
  "success": true,
  "data": {
    "analytics": {
      "revenue_trends": "object",
      "user_engagement": "object",
      "subscription_metrics": "object",
      "attendance_patterns": "object"
    },
    "insights": "array"
  }
}
```

Authentication: JWT + role('admin')

8. GET /api/admin/export

Purpose: Export system data

Input: Query parameters: type, format, start_date, end_date

Output:

```
{
  "success": true,
  "message": "Export prepared successfully",
  "data": {
    "downloadUrl": "string",
    "filename": "string",
    "size": "number"
  }
}
```

Authentication: JWT + role('admin')

9. GET /api/admin/audit-logs

Purpose: Get audit trail logs

Input: Query parameters: action, user_id, start_date, end_date, page, limit

Output:

```
{
  "success": true,
  "data": {
    "logs": [
      {
        "id": "number",
        "action": "string",
        "performed_by": "string",
        "target": "string",
        "timestamp": "datetime",
        "details": "object"
      }
    ],
    "pagination": {}
  }
}
```

Authentication: JWT + role('admin')

10. GET /api/admin/health

Purpose: Get system health status

Input: None

Output:

```
{
  "success": true,
  "data": {
    "status": "healthy",
    "uptime": "number",
    "database": {
      "status": "connected",
      "response_time": "number"
    },
    "services": {
      "email": "operational",
      "notifications": "operational"
    },
    "metrics": {
      "memory_usage": "number",
      "cpu_usage": "number",
      "active_connections": "number"
    }
  }
}
```

Authentication: JWT + role('admin')

11. POST /api/admin/backup

Purpose: Create system backup

Input:

```
{
  "type": "string (optional, 'full' or 'incremental', default: 'full')",
  "include_files": "boolean (optional, default: false)"
}
```

Output:

```
{
  "success": true,
  "message": "Backup initiated successfully",
  "data": {
    "backup_id": "string",
    "estimated_completion": "datetime",
    "size_estimate": "string"
  }
}
```

Authentication: JWT + role('admin')

OAuth Routes (/api/oauth)

1. GET /api/oauth/google

Purpose: Initiate Google OAuth authentication

Input: None (redirects to Google)

Output: HTTP redirect to Google OAuth consent screen

Authentication: None required

2. GET /api/oauth/google/callback

Purpose: Handle Google OAuth callback

Input: Query parameters from Google (code, state, etc.)

Output:

```
{
  "success": true,
  "message": "Google authentication successful",
  "token": "jwt_token_string",
  "user": {
    "id": "number",
    "name": "string",
    "email": "string",
    "role": "string",
    "status": "string",
    "email_verified": true,
    "profile_picture": "string"
  }
}
```

Authentication: None required (handled by Passport)

Side Effects:

- Creates new user if not exists
- Links Google account to existing user
- Sets HTTP-only auth cookie
- Sends welcome email (async)
- Creates notification (async)

3. GET /api/oauth/google/failure

Purpose: Handle Google OAuth failure

Input: None

Output:

```
{
  "success": false,
  "message": "Google authentication failed",
  "error": "Authentication was cancelled or failed"
}
```

Authentication: None required

Rate Limiting

The API implements different rate limiting strategies:

- **Register Limiter:** Stricter limits for registration endpoints
- **Auth Limiter:** Moderate limits for login attempts
- **Email Verification Limiter:** Limits for email verification
- **General Limiter:** Standard limits for most endpoints

Middleware

Authentication Middleware

- `verifyToken`: Validates JWT tokens
- `authorize(roles...)`: Checks user roles

Validation Middleware

- Input validation using `express-validator`
- Custom validation services for different entities

Error Handling

All endpoints follow consistent error response format:

```
{
  "success": false,
  "message": "Error description",
  "error": "Detailed error information (in development)"
}
```

Notable Limitations

1. **Settings Management:** Not implemented - requires Settings model
2. **Class Scheduling:** Limited due to missing date fields in Class model
3. **User Preferences:** Not implemented - requires schema changes
4. **Profile Pictures:** Removed due to complexity
5. **Advanced Reporting:** Limited by current model relationships

This report covers all implemented routes and their complete input/output specifications as of the current implementation.